

Operating System

Chapter 1 & 2 — Question and Answer Bank

Based on: Silberschatz, Galvin & Gagne, 10th Edition

Section A: Exam Questions (From Question Paper)

Question 1

Q: What is the purpose of interrupts? How does an interrupt differ from a trap? Discuss the interrupt-driven I/O cycle with figure.

Answer:

Purpose of Interrupts:

An interrupt is a signal sent by a hardware device or software to the CPU to inform it that an event needs immediate attention. The CPU stops its current task, saves its state, handles the interrupt by running the Interrupt Service Routine (ISR), and then resumes the original task.

The main purpose of interrupts is to allow the CPU to perform other tasks while waiting for slow I/O devices. Without interrupts, the CPU would waste time polling devices to check if they are ready.

Interrupt vs. Trap:

Feature	Explanation
Interrupt	Generated by hardware devices (e.g., keyboard, disk). External event.
Trap	Generated by software — either a program error (exception) or a deliberate request for OS service (system call). Internal event.
Example of Interrupt	Disk read is complete; printer finishes a page.
Example of Trap	Division by zero (exception); program calls open() to read a file (system call).

Interrupt-Driven I/O Cycle (Step by Step):

- Step 1: The device driver in the OS initiates the I/O operation (e.g., requests a disk read).
- Step 2: The I/O controller starts the operation on the device.
- Step 3: The CPU continues executing other tasks while the device works.
- Step 4: When the device finishes, it sends an interrupt signal to the CPU.
- Step 5: The CPU receives the interrupt, finishes the current instruction, and transfers control to the interrupt handler.
- Step 6: The interrupt handler processes the data (e.g., moves it from the device buffer to memory).
- Step 7: The CPU restores its saved state and resumes the interrupted program.

This cycle allows the CPU to remain productive during slow I/O operations, improving overall system efficiency.

Question 2

Q: Identify several advantages and several disadvantages of open-source operating systems. Identify the types of people who would find each aspect to be an advantage or a disadvantage.

Answer:

Advantages of Open-Source Operating Systems:

Advantage	Who Benefits
Free of cost — no license fee	Students, individuals, small businesses, and universities with limited budgets benefit greatly.
Source code is publicly available	Researchers, security professionals, and developers can study, audit, and verify the code.
Anyone can modify and customize	System administrators and developers can tailor the OS to meet specific needs.
Community-driven development and fast bug fixes	General users and enterprise organizations benefit from rapid updates.
High transparency — no hidden backdoors	Governments, financial institutions, and security-sensitive organizations find this valuable.

Disadvantages of Open-Source Operating Systems:

Disadvantage	Who Is Affected
Limited official support — no company helpdesk	Non-technical users and businesses that depend on guaranteed technical support find this a serious drawback.
Steeper learning curve for beginners	General home users accustomed to Windows or macOS may find Linux difficult.
Hardware compatibility issues	Casual users and gamers may face driver problems for newer hardware.
Fragmentation — many distributions exist	Businesses deploying across teams may struggle with inconsistency between distributions.
Exposed source code could help attackers find vulnerabilities	Some security teams consider this a risk, though open review often leads to faster fixes.

Question 3

Q: What is Direct Memory Access (DMA), and how does it improve system performance by reducing CPU intervention during data transfers between high-speed I/O devices and main memory?

Answer:

Definition of DMA:

Direct Memory Access (DMA) is a feature that allows certain hardware devices to transfer data directly to and from main memory (RAM) without involving the CPU for every byte of data. A dedicated DMA controller manages this transfer.

How DMA Works:

- Step 1: The CPU instructs the DMA controller by providing the source address (device buffer), destination address (RAM location), and number of bytes to transfer.
- Step 2: The CPU moves on to execute other tasks. The DMA controller takes control of the memory bus.
- Step 3: The DMA controller transfers the entire block of data directly between the device and memory.
- Step 4: When the transfer is complete, the DMA controller sends a single interrupt to the CPU to inform it.
- Step 5: The CPU resumes handling the data.

How DMA Improves System Performance:

Without DMA	With DMA
CPU reads or writes one byte at a time	DMA controller transfers the entire block at once
One CPU interrupt per byte — very inefficient	Only one interrupt per block transfer
CPU is fully occupied during I/O	CPU is free to do other work during the transfer
Poor performance for large data transfers	High performance for disk, network, and USB transfers

DMA is commonly used in disk drives, network interface cards, sound cards, and USB controllers. It is essential for any high-speed I/O operation where transferring large blocks of data is required.

Question 4

Q: What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach?

Answer:

Main Advantage of the Microkernel Approach:

The main advantage of the microkernel approach is reliability and stability. In this design, only the most essential functions — **inter-process communication (IPC), basic memory management, and CPU scheduling** — remain inside the kernel. All other services such as file systems and device drivers run in user mode as separate processes. If one service crashes, it does not affect the kernel or other services. The system remains running.

How User Programs and System Services Interact:

In a microkernel architecture, user programs do not call kernel services directly. Instead, they communicate through message passing. The flow is as follows:

- A user program sends a message to the microkernel requesting a service (for example, reading a file).
- The microkernel forwards the message to the appropriate user-mode service process (for example, the file system server).
- The file system server processes the request and sends a reply message back through the microkernel.
- The microkernel delivers the response to the user program.

All communication between user programs and services passes through the microkernel using messages. No direct calls to kernel-level code are made by the services.

Examples of microkernel-based systems include Mach (which forms the base of macOS Darwin), QNX, L4, and Minix.

Disadvantages of the Microkernel Approach:

- Performance overhead: Because every service request requires multiple context switches and message-passing operations between user space and kernel space, the system becomes slower compared to a monolithic kernel.
- Higher latency: The message-passing mechanism adds communication delay, which can be noticeable in performance-critical applications.
- Increased complexity: Designing reliable inter-process communication for all system services is complex and difficult to implement correctly.

Section B: Additional Tutorial Questions and Answers

Tutorial Question 1

Q: What is an Operating System? What are its two main roles?

Answer:

An Operating System (OS) is a special program that acts as an intermediary between the user and the computer hardware. It runs at all times while the computer is on.

Its two main roles are:

- Resource Allocator: It manages all hardware resources — CPU time, memory, disk space, and I/O devices — and allocates them to programs as needed.
- Control Program: It controls the execution of user programs to prevent errors and misuse, protecting the system from crashes or unauthorized access.

Tutorial Question 2

Q: Explain the difference between multiprogramming and multitasking.

Answer:

Feature	Explanation
Multiprogramming	Multiple programs are kept in memory at the same time. When the running program waits for I/O, the CPU switches to another program. The goal is to keep the CPU busy at all times. Used in batch processing systems.
Multitasking (Time-Sharing)	The CPU switches between programs so rapidly (many times per second) that each user feels they have the CPU to themselves. Response time is less than one second. Used in interactive systems like Windows and Linux.

In short: multiprogramming focuses on CPU utilization, while multitasking focuses on fast interactive response for users.

Tutorial Question 3

Q: What is a system call? Give two examples. Why do programs use APIs instead of calling the kernel directly?

Answer:

A system call is the programmatic interface through which a user program requests a service from the Operating System kernel. It provides a controlled and safe way to switch from User Mode to Kernel Mode, where privileged hardware operations can be performed.

Examples of system calls:

- `open()` and `read()` in Linux — used to open and read files.
- `fork()` in Linux — used to create a new process.

Why programs use APIs instead of calling the kernel directly:

Programs use high-level Application Programming Interfaces (APIs) such as the POSIX API or Win32 API rather than invoking system calls directly because:

- APIs provide a portable and programmer-friendly interface. The same API code can work across different OS versions without change.
- APIs hide the low-level details of system call numbers and mode switching from the programmer.
- Libraries such as `libc` handle the conversion from the API call to the actual kernel system call internally.

Tutorial Question 4

Q: What is the purpose of Dual-Mode operation in an OS? What is the mode bit?

Answer:

Dual-mode operation protects the Operating System and hardware from being damaged or misused by faulty or malicious user programs.

The CPU operates in two modes:

- User Mode: All user applications run here. Programs cannot directly access hardware or execute privileged instructions. The mode bit is set to 1.
- Kernel Mode: Only the OS runs here. It has full access to all hardware and can execute any instruction. The mode bit is set to 0.

The mode bit is a single bit in the CPU's hardware registers that indicates the current mode. When a user program makes a system call, the CPU switches to Kernel Mode (mode bit = 0). After the OS finishes handling the request, it returns to User Mode (mode bit = 1).

This mechanism ensures that user programs cannot directly manipulate hardware or interfere with the OS.

Tutorial Question 5

Q: What is the storage hierarchy? Why can the CPU only directly use RAM?

Answer:

The storage hierarchy is an arrangement of different types of storage from fastest and smallest at the top to slowest and largest at the bottom. The levels are:

- Registers — inside the CPU, fastest, very small.

- Cache Memory — slightly slower than registers, still very fast.
- Main Memory (RAM) — fast, volatile (data is lost when power is off).
- Nonvolatile Memory (SSD/NVM) — fast, non-volatile secondary storage.
- Hard Disk Drive (HDD) — slow, large, non-volatile.
- Optical Disk and Magnetic Tape — very slow, used for backup.

The CPU can only directly execute instructions and access data that are in main memory (RAM). Data stored on a hard disk or SSD cannot be used by the CPU until it is first copied into RAM. This is because RAM is directly connected to the CPU through the memory bus and can be addressed by the CPU directly. Disk storage requires a separate I/O operation to move data into RAM first.

Tutorial Question 6

Q: Compare the monolithic kernel and microkernel approaches to OS design.

Answer:

Feature	Details
Monolithic Kernel	All OS services — file system, device drivers, memory management, CPU scheduling — run together inside the kernel in a single large block. All components can communicate directly.
Microkernel	Only the most essential functions (IPC, basic memory, CPU scheduling) remain in the kernel. All other services run in user mode as separate processes.
Speed	Monolithic is faster because services communicate through direct function calls. Microkernel is slower due to message-passing overhead.
Stability	In a monolithic kernel, a bug in any service can crash the entire system. In a microkernel, a service crash does not affect the kernel or other services.
Security	Microkernel is more secure because user-mode services have no privileged access.
Examples	Linux, early UNIX (monolithic). Mach, QNX, Minix (microkernel).

Tutorial Question 7

Q: What is Spooling? Give a practical example and explain its main benefit.

Answer:

Spooling stands for Simultaneous Peripheral Operations OnLine. It is the process of storing data intended for a slow I/O device (such as a printer) in a buffer — a special area on disk or in memory — so that the CPU does not have to wait for the device to finish before continuing with other work.

How it works:

- A program sends output data to the OS instead of directly to the device.
- The OS stores this data in a spool buffer on disk.

- The CPU immediately moves on to execute other tasks.
- The slow device (such as a printer) reads from the spool buffer at its own pace.

Practical Example:

In a university computer lab, if ten students send documents to print at the same time, all ten jobs are placed in a print queue (spool). The printer picks and prints them one by one. Each student can continue working on the computer immediately without waiting.

Main Benefit:

Spooling allows the CPU and the I/O device to work in parallel. The CPU is not kept idle waiting for a slow device. This significantly improves CPU utilization and overall system throughput.

Tutorial Question 8

Q: What is the role of the Bootstrap program? Where is it stored and what does it do?

Answer:

The Bootstrap program, also called the bootloader, is the very first program that runs when a computer is powered on. It is stored in ROM (Read-Only Memory) or EEPROM on the motherboard, not on the hard disk, so it is available immediately when power is applied.

The Bootstrap program performs the following steps:

- It initializes the CPU registers and hardware components.
- It checks that the hardware (RAM, keyboard, storage) is functioning correctly.
- It locates the Operating System kernel stored on the disk.
- It loads the OS kernel into RAM.
- It transfers control to the kernel, which then takes over management of the entire system.

After the kernel starts, it launches system daemons (background services) and eventually presents the login screen to the user.

Tutorial Question 9

Q: What are OS design goals and policies? What is the difference between policy and mechanism in OS design?

Answer:

OS Design Goals:

When designing an OS, two broad sets of goals must be considered:

- User Goals: The OS should be convenient to use, easy to learn, reliable, safe, and fast from the user's perspective.
- System Goals: The OS should be easy to design, implement, and maintain. It should be flexible, reliable, error-free, and efficient from the system designer's perspective.

Policy vs. Mechanism:

The separation of policy and mechanism is one of the most important principles in OS design.

- Policy answers the question: What should be done? It defines the rules and decisions. For example: "Which process should get the CPU next?"

- Mechanism answers the question: How should it be done? It defines the tools and methods that implement the policy. For example: “Use a timer to switch the CPU between processes.”

Why this separation matters: If policies and mechanisms are kept separate, then the policy can be changed without rewriting the mechanism. For example, if the CPU scheduling policy changes from Round Robin to Priority Scheduling, the underlying timer mechanism does not need to change. This makes the OS flexible and easier to maintain.

Real-life example: A traffic rule that says “give ambulances priority” is a policy. The traffic signal system that enforces it is the mechanism. The signal system can also enforce other rules without being rebuilt.

Tutorial Question 10

Q: What are the different types of OS user interfaces? Compare Command Line Interface (CLI) with Graphical User Interface (GUI).

Answer:

Types of OS User Interfaces:

The Operating System provides a user interface (UI) so that users and programs can communicate with it. There are three main types:

- Command Line Interface (CLI): The user types text commands directly. The OS reads the command, executes it, and displays the result as text. Example: Linux Terminal, Windows Command Prompt.
- Graphical User Interface (GUI): The user interacts using visual elements such as icons, windows, and menus through a mouse or touch screen. Example: Windows Desktop, macOS Finder, Android home screen.
- Touch-Screen Interface: Used on mobile devices. Input is given through finger gestures such as tap, swipe, and pinch. Example: Android, iOS.

CLI vs. GUI Comparison:

Feature	CLI	GUI
Input Method	Text commands typed by keyboard	Mouse clicks on icons and menus
Ease of Use	Harder for beginners; requires memorizing commands	Easy for beginners; visual and intuitive
Speed for Experts	Faster; complex tasks done in one command or script	Slower for complex tasks; requires many mouse clicks
Resource Usage	Uses very little memory and CPU	Uses more memory and CPU for graphics rendering
Automation	Excellent; tasks can be scripted and automated easily	Limited automation; mostly manual interaction required
Examples	Linux Bash, Windows CMD, PowerShell	Windows Desktop, macOS Finder, GNOME on Linux

Most modern operating systems provide both a CLI and a GUI. For example, Linux provides a graphical desktop environment (GNOME or KDE) as well as a powerful terminal (CLI) for advanced users. Users can choose the interface that best suits their task.